Vue Fundamentals

Module – Data and bindings

Peter Kassenaar –
info@kassenaar.com

For visuals – add Bootstrap

- There are (a lot!) of other Vue-optimized UI-frameworks
  - We can cover them later
  - We now just add default Bootstrap to get some basic styling

https://getbootstrap.com/

Adding Bootstrap to a Vue project

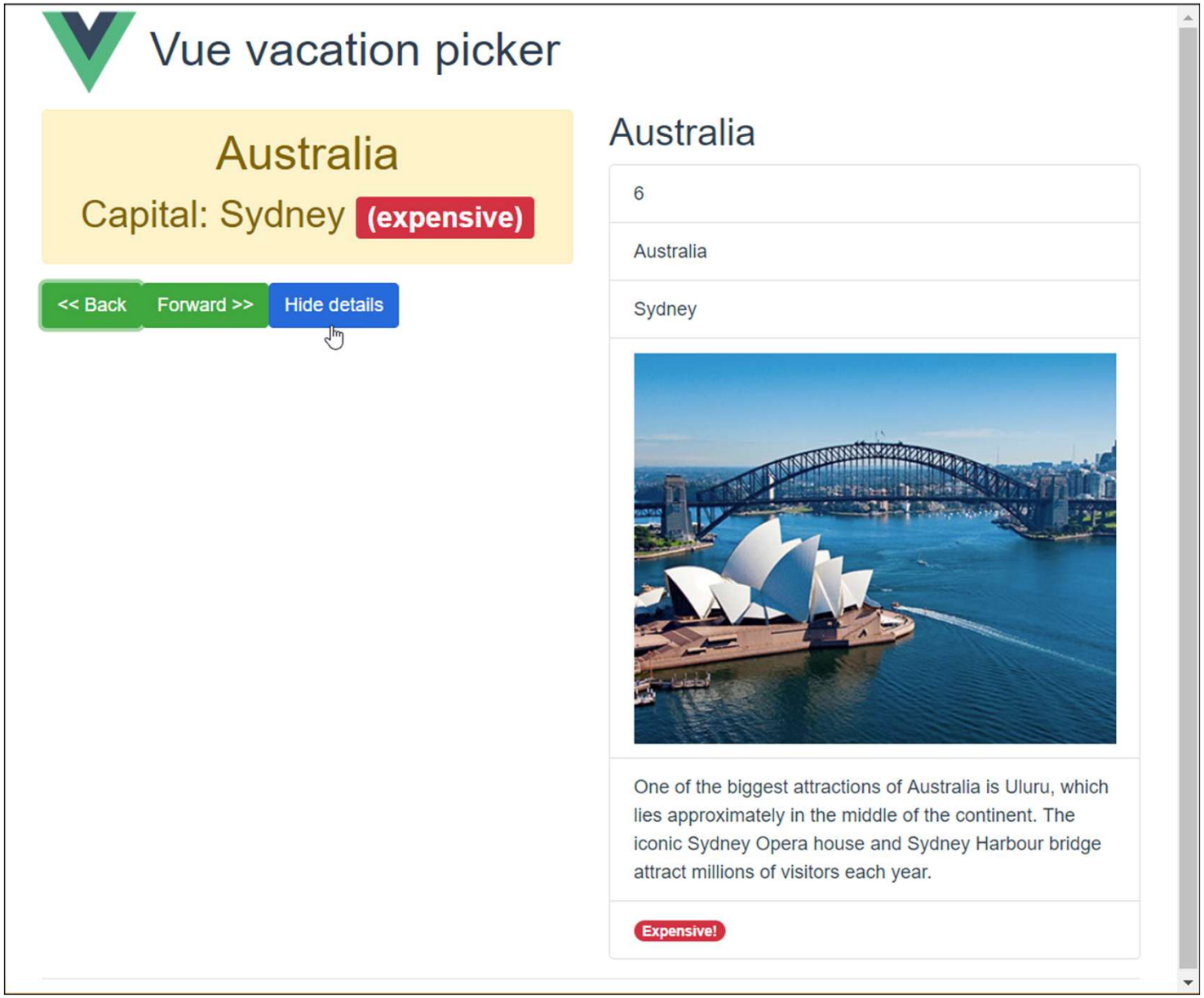
```
npm install bootstrap [--save]
```

- `Bootstrap` for basic styling and components
- NO NEED TO INSTALL jQuery and popper in Bootstrap 5+
- The --save flag is optional since NPM 5

In `main.js`, add:

```
// Bootstrap styling
import 'bootstrap';
import 'bootstrap/dist/css/bootstrap.min.css';
```

What are we going to build



Data binding

- Vue knows four ways of data binding:
  - Simple Data binding Expressions: `{{ … }}`
  - Attribute binding with `v-bind:`
  - Event binding with `v-on:`
  - Two-way binding with `v-model`

Building an app: VactionPicker

- We're going to build a simple vacation picker.
  - Data comes from a separate `.js`-file (but will come from a real db in the future!)
- Requirements
  1. User can cycle through different destination countries
  2. User can show/hide details for each destination
  3. [User can add trips to countries to a shopping cart
  4. Shopping cart can be send to the backend for processing]

1. Creating the data file

```
// data.js - holding an array of country/capital data.
// This of course will come from a real db in the future.
const data = {
   countries: [
      {
         id: 1,
         name: 'USA',
         capital: 'Washington',
         cost: 1250,
         details: 'United States are among the most visited country in the world.',
         img: 'washington.jpg'
      },
      {
         id: 2,
         name: 'Netherlands',
         capital: 'Amsterdam',
         cost: 795,
         details: 'The capital of the Netherlands, Amsterdam, is over 1000 years old.',
         img: 'amsterdam.jpg'
      },
      { …
}
```

Our example: a simple JavaScript object, holding an array with countries and some (fake) data, don't forget to `export` it!

2. Creating components

Create a new component, `VacationPicker.vue` with default content.

```
<template>
    <div class="">
        <h1>Vacation Picker</h1>
    </div>
</template>


<script setup>


</script>
```

3. Importing data

- Use default `import` statement for the `data.js` file
- Add the data to a variable in the script

```
<script setup>
  // import the country countryData
  import countryData from '@/data/CountryData';


  // create variables
  const data = countryData;
</script>
```

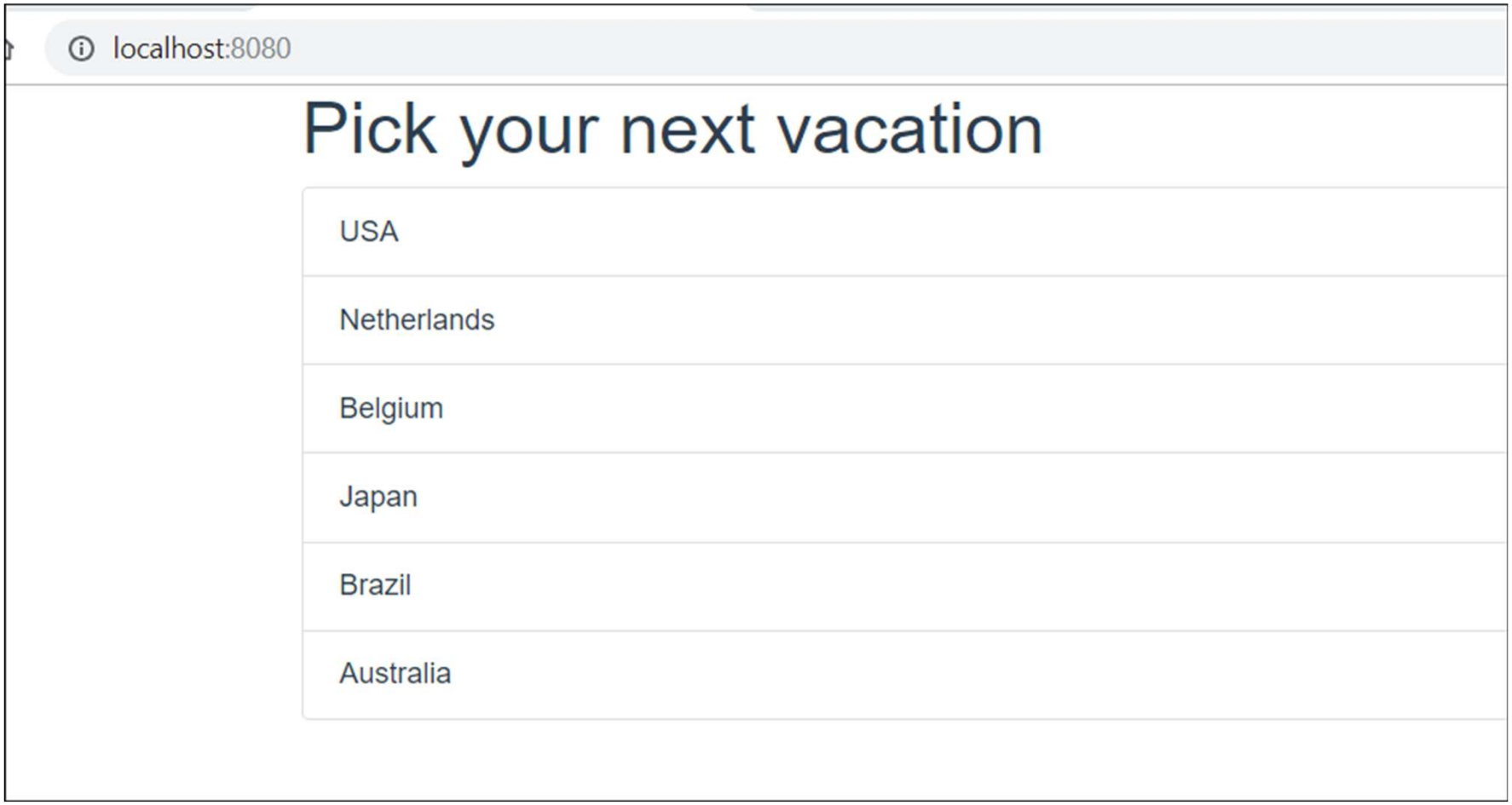
4. Binding to the data

Use the `v-for` directive to render a list of items, based on an array

```
<ul class="list-group">
    <li class="list-group-item" v-for="country in data.countries">
       {{country.name}}
    </li>
</ul>
```

https://vuejs.org/v2/guide/list.html

5. Result

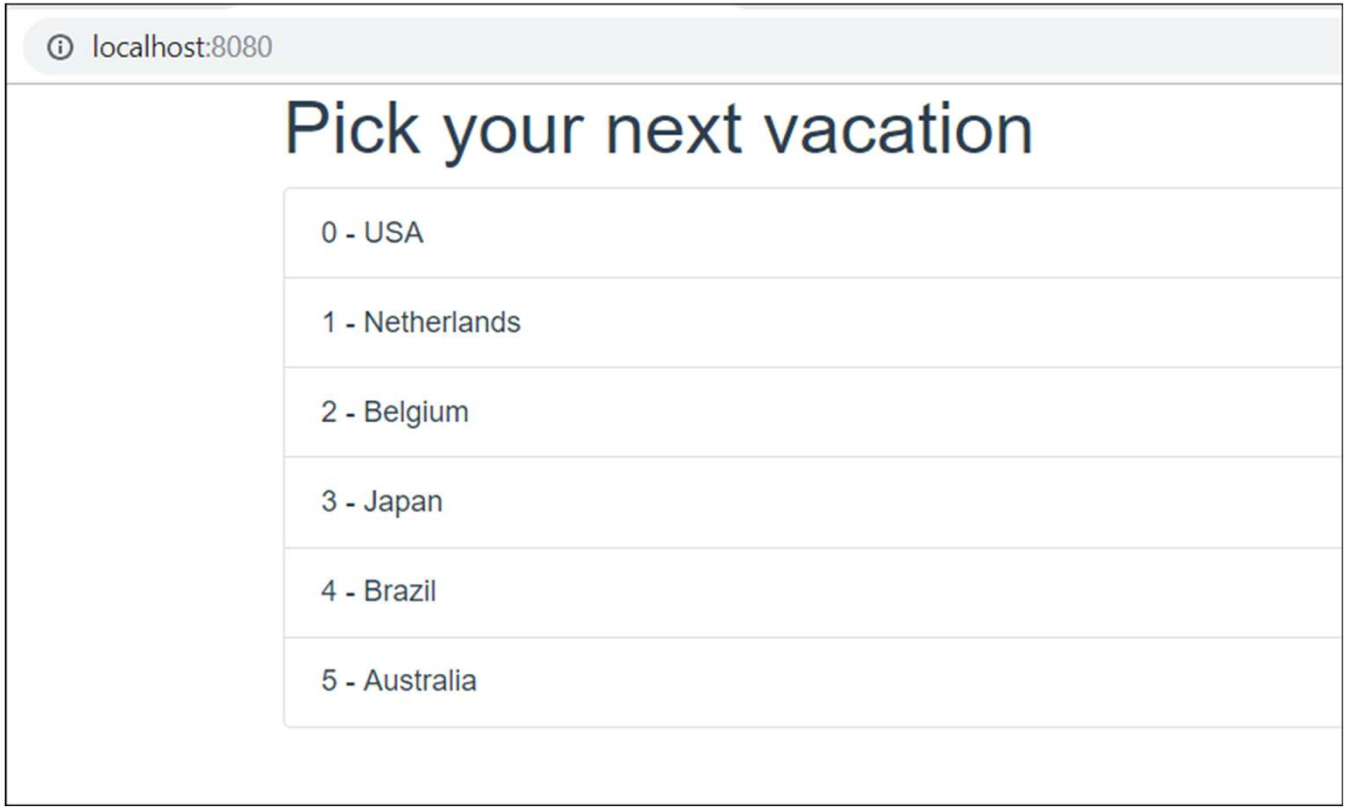


(assumes you updated `App.vue` to include the `VacationPicker` component and update/remove some styles)

Data binding expression

- Inside `v-for` blocks we have full access to parent scope properties.
  - Use the data binding expression `{{ … }}` to bind to properties
- `v-for` also supports an optional second argument for the `index` of the current item.

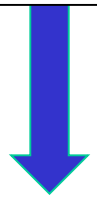
```
<li v-for="(country, index) in data.countries">
   {{ index }} - {{country.name}}
</li>
```



More data binding

- You can bind to *any property* by assigning a variable

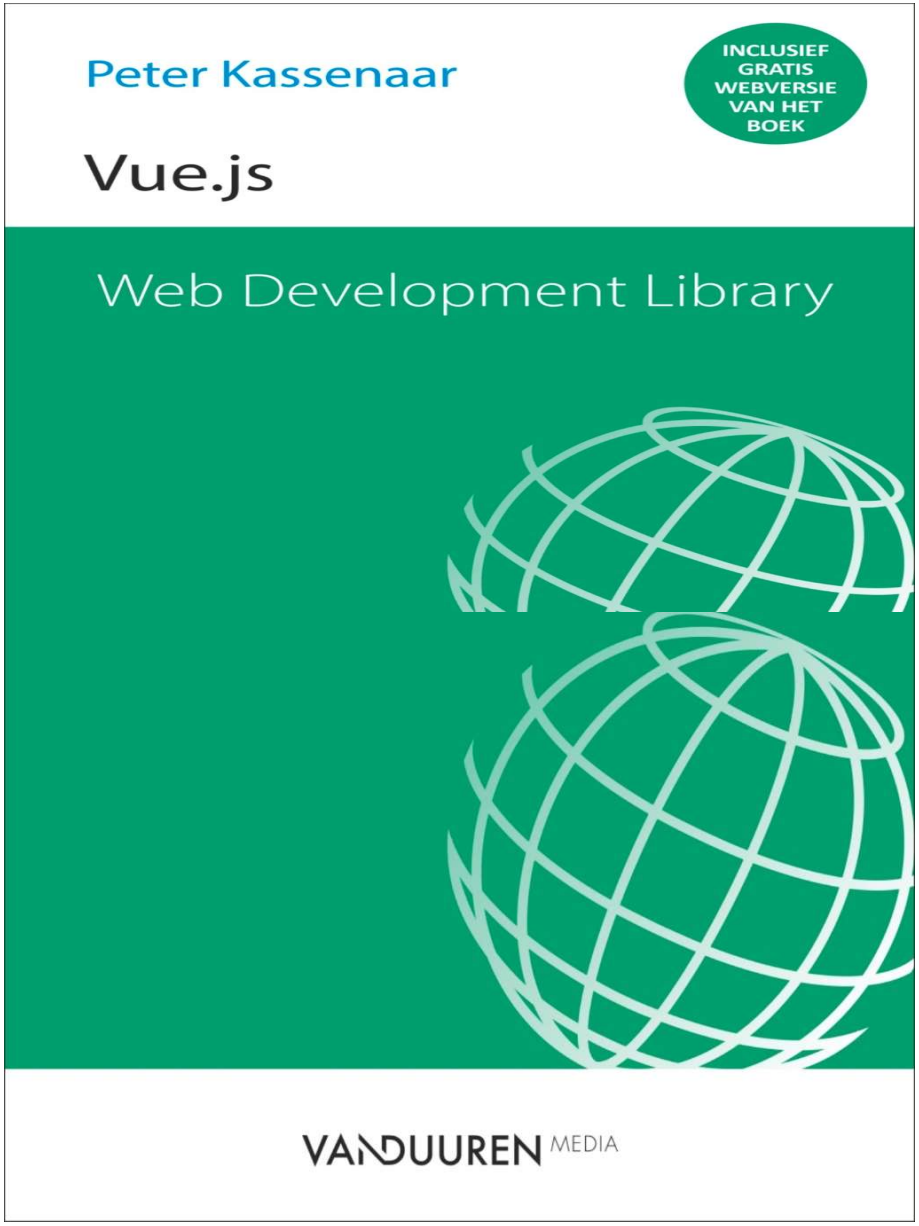
```
const header = 'Pick your next vacation';
```



```
<h1>{{ header }}</h1>
```

Using v-bind

Creating dynamic attributes on HTML- and custom elements

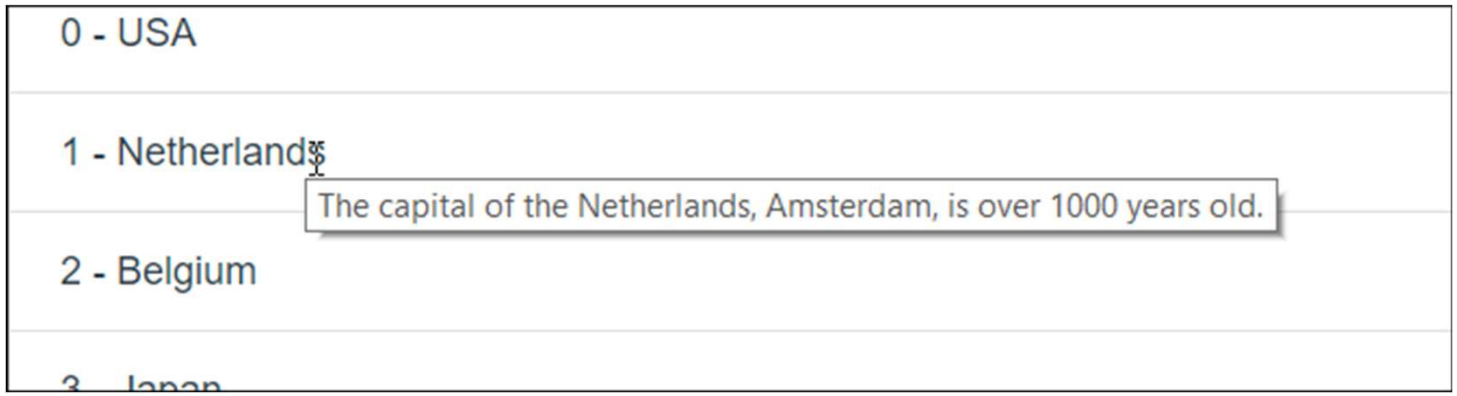

Peter Kassenaar
INCLUSIEF GRATIS WEBVERSIE VAN HET BOEK
Vue.js
Web Development Library
VANDUUREN MEDIA

P. 87 e.v.

Used more often v-bind:

- Add `v-bind:` as prefix to an HTML-property to set it's content
- If you want to add an `id` and `title` attribute for example:

```
<span v-bind:id="country.id"
      v-bind:title="country.details">
    {{ index}} - {{country.name}}
</span>
```



Lists: using the `v-bind:key` property

```
<ul>
  <li v-for="country in countries.countries">
    {{ country.id}} - {{ country.name }}</li>
</ul>
</div>
```

The editor is complaining it is missing a `v-bind:key` directive here

```
4     <ul>
5       <li v-for="country in countries.countries">
6
7     </
8   </di
9 </temp
0 <scrip
1 import
2
3 export
```

[vue/require-v-for-key]
Elements in iteration expect to have 'v-bind:key' directives. eslint-plugin-vue

Elements in iteration expect to have 'v-bind:key' directives. (vue/require-v-for-key) eslint

Quick Fix... Peek Problem

Why the `key` attribute?

- Create a dynamic `key` attribute for each element in a repeated collection
- Vue needs to be able (as other frameworks) to uniquely identify each element in a list
  - Shadow DOM! Or Virtual DOM
- Solution: use the `v-bind:key` attribute

```
<li class="list-group-item"
    v-for="country in data.countries"
    v-bind:key="country.id">
```

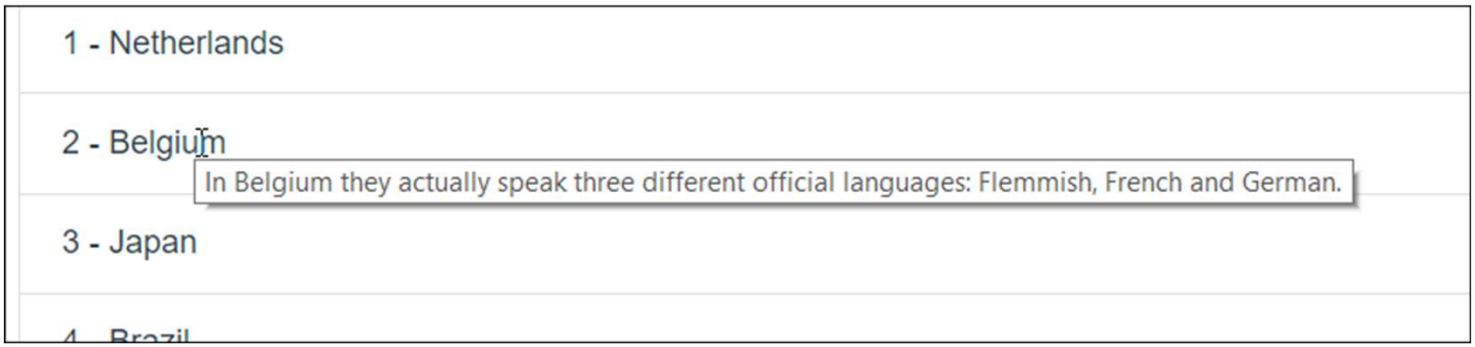
Summary

The `v-bind:` notation can be used on any attribute and it is used every time you need the attribute to be dynamic (e.g. adjusted from within code)

Using shorthand notation :

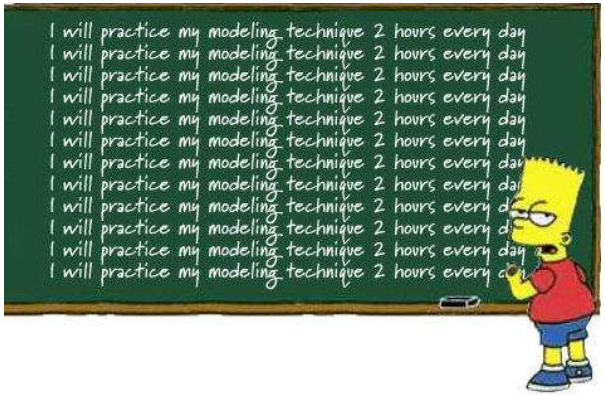
Same effect: using just `:` as a shorthand notation for `v-bind`:

```
<span :id="country.id"
      :title="country.details">
    {{ index}} - {{country.name}}
</span>
```



Workshop

- Add a data file to your application.
  - You can use `CountryData.js` as an example, or build your own data file
  - Use for instance `CustomerData.js` as a blueprint and add your own data
- Import the data file to your application
  - Assign the data to a variable
- Show its contents in a new component, or inside `VacationPicker`, using `v-for`
- Create a `v-bind:key` binding for your data
- Example: `../103-databinding`
- Optional: Experiment with `v-text` and `v-html`



Using event binding

Handling events from the user interface

Peter Kassenaar
INCLUSIEF GRATIS WEBVERSIE VAN HET BOEK
Vue.js
Web Development Library
VANDUUREN MEDIA

P. 92 e.v.

Creating a `ref()`

- When the variable needs to be updated, we use the `ref()` function
  - `ref()` only in Composition API!
- This makes the variable reactive.
  - Whenever its `value` changes, the view is rerendered automatically

```
<script setup>
import {ref} from "vue";

const counter = ref(0);

//…

</script>
```

Declaring Reactive State

```js
import { ref } from 'vue'

const count = ref(0)
```

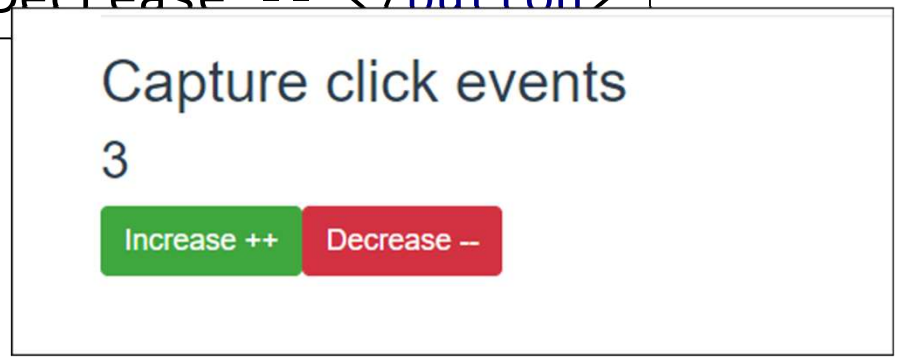
`ref()` takes the argument and returns it wrapped within a ref object with a `.value` property:

```js
const count = ref(0)

console.log(count) // { value: 0 }
console.log(count.value) // 0

count.value++
console.log(count.value) // 1
```

Binding events with v-on

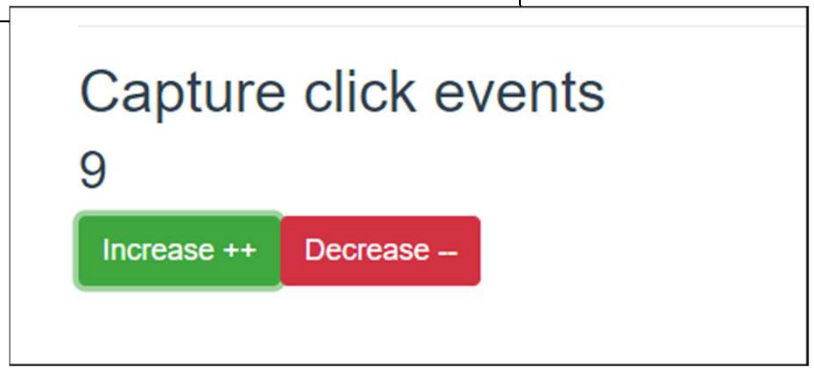
- You can bind to DOM-events using the `v-on:` syntax
  - Events like `click`, `blur`, `focus`, `mouseover`, etc
  - https://developer.mozilla.org/en-US/docs/Web/Events
  - (add the `counter` variable to the `data` property)

```
<h3>{{ counter }}</h3>
<button v-on:click="counter++">Increase ++ </button>
<button v-on:click="counter--">Decrease -- </button>
```



Or, use the shorthand notation @

```
<h3>{{ counter }}</h3>

<button @click="counter++">Increase ++ </button>

<button @click="counter--">Decrease -- </button>
```



More likely, you'll be calling a function

- Functions for components are called `methods`
- They are defined on the component script block
- Methods can accept parameters, like any other function
- Remember to use the `value` property when updating a reactive variable: `<varName>.value`

```
<template>
  <div class="">
    <h3>{{ counter }}</h3>
    <button @click="increase()" class="btn btn-success">Increase ++</button>
    <button @click="decrease()" class="btn btn-danger">Decrease --</button>
  </div>
</template>

<script setup>
import {ref} from "vue";

const counter = ref(0);

// functions
const increase = () => {
  counter.value++;
}

const decrease = () => {
  counter.value--;
}
</script>
```

```
// classical notation
function increase() {
  counter.value++;
}

function decrease() {
  counter.value--;
}
```

Use whatever notation you like,

but be consistent

Information on Reactivity

Vue.js Search Ctrl K

Docs API Playground Ecosystem About Sponsor Experts NEW

API Preference

Options Composition

Getting Started
Introduction
Quick Start

Essentials
Creating an Application
Template Syntax
Reactivity Fundamentals
Computed Properties
Class and Style Bindings
Conditional Rendering
List Rendering
Event Handling
Form Input Bindings
Lifecycle Hooks
Watchers
Template Refs
Components Basics

Components In-Depth
Registration
Props
Events
Component v-model

Reactivity Fundamentals

API Preference

This page and many other chapters later in the guide contain different content for the Options API and the Composition API. Your current preference is **Composition API**. You can toggle between the API styles using the "API Preference" switches at the top of the left sidebar.

Declaring Reactive State

`ref()`

In Composition API, the recommended way to declare reactive state is using the `ref()` function:

```js
import { ref } from 'vue'

const count = ref(0)
```

`ref()` takes the argument and returns it wrapped within a ref object with a `.value` property:

ON THIS PAGE

Declaring Reactive State
ref()
<script setup>
Why Refs?
Deep Reactivity
DOM Update Timing
reactive()
Reactive Proxy vs. Original
Limitations of reactive()
Additional Ref Unwrapping Details
As Reactive Object Property
Caveat in Arrays and Collections
Caveat when Unwrapping in Te...

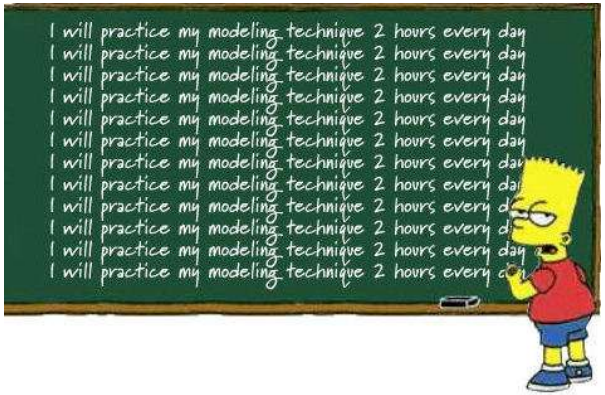
Vue Mastery Vue School VEHIKL PASSIONATE PEOPLE storyblok herodevs JS JavaScript Certification
Become a Sponsor

https://vuejs.org/guide/essentials/reactivity-fundamentals.html

Workshop

1. Add a button to the page that toggles the `disabled` attribute of another button
    1. Tip: create a dynamic `:disabled="isDisabled"` property
2. When a specific country in the list is clicked, show it's name and capital in a JavaScript `alert()` box (or use `console.log`)
3. Log the name of the current country and it's index/id to the console on a mouseover

- Use `v-on:` and `@`-notation
- Example: `../104-eventbinding`



Computed properties

Binding to more complex properties and update them on changing

Peter Kassenaar
INCLUSIEF GRATIS WEBVERSIE VAN HET BOEK
Vue.js
Web Development Library
VANDUUREN MEDIA

P. 104 e.v.

Selecting a specific country

- We want to select a `country` (or `product`, or `employee`, or whatever) when a user clicks on them
- Create a `computed(…)` variable on the component that composes and returns the requested data
  - Here: we're setting a property that holds the currently selected index
  - Update the computed property when the index changes
- Computed properties avoid having to do complex computations inside the HTML-view!
- A computed property is only evaluated once its dependencies change

1. In the view

```
<li @click="selectCountry(country.id)"
    class="list-group-item" v-for="country in data.countries">
```

2. On the component

```
import {computed, ref} from "vue";
import countryData from '@/data/CountryData';

// Automatically calculate current country, based on Id
const currentCountry = computed(()=>{
  return data.countries.find(c => c.id === currentCountryId.value)
})

// Update the Id when the user clicks a country
const selectCountry = id => {
  currentCountryId.value = id;
}
```

3. On the component again

```
<h2>Selected:</h2>
<ul class="list-group">
    <li class="list-group-item">{{ selectedCountry.id}}</li>
    <li class="list-group-item">{{ selectedCountry.name}}</li>
    <li class="list-group-item">{{ selectedCountry.capital}}</li>
    <li class="list-group-item">{{ selectedCountry.details}}</li>
</ul>
```

4. Result

Computed properties or methods?

"Instead of a computed property, we can define the same function as a function instead. For the end result, the two approaches are indeed exactly the same. However, the difference is that **computed properties are cached based on their dependencies.** A computed property will only re-evaluate when some of its dependencies have changed. "

https://vuejs.org/guide/essentials/computed.html

`v-if` and more computed properties

- Show a badge when a destination is expensive
- Use `v-if` to render a DOM element if a certain condition is `true`
  - You can use `v-else` and `v-else-if` to render elements (though used less often)

```
<li class="list-group-item" v-if="isExpensive">
    <span class="badge bg-danger rounded-pill">Expensive!</span>
</li>
```

```
// Automatically calculate if a destination is expensive
const isExpensive = computed(() => {
  return currentCountry.value.cost > 4000;
});
```

https://vuejs.org/v2/guide/conditional.html

Read-only

- Treat computed properties as *read-only* variables
- They are a snapshot. It doesn't make sense to mutate a snapshot.

Best Practices

Getters should be side-effect free

It is important to remember that computed getter functions should only perform pure computation and be free of side effects. For example, **don't mutate other state, make async requests, or mutate the DOM inside a computed getter!** Think of a computed property as declaratively describing how to derive a value based on other values - its only responsibility should be computing and returning that value. Later in the guide we will discuss how we can perform side effects in reaction to state changes with watchers.

Avoid mutating computed value

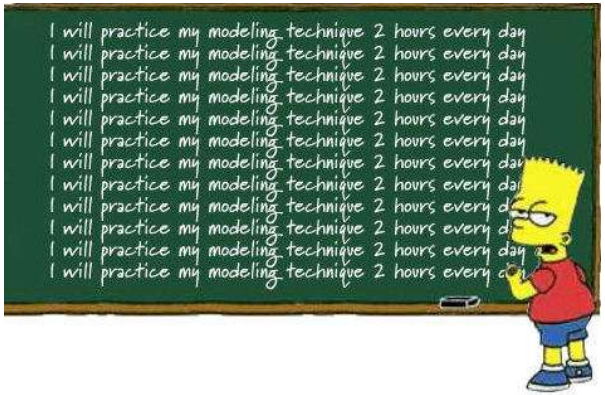
The returned value from a computed property is derived state. Think of it as a temporary snapshot - every time the source state changes, a new snapshot is created. It does not make sense to mutate a snapshot, so a computed return value should be treated as read-only and never be mutated - instead, update the source state it depends on to trigger new computations.

https://vuejs.org/guide/essentials/computed.html#best-practices

Workshop

1. Create a detail view for your own app.
    1. If an element/data is clicked, show details in the UI
2. Create a computed property that show a destination as 'on sale' if the cost is less than 1000.
3. Replace the text `Selected` with the actual name of the selected country

- Use `v-if` where appropriate
- Example: `../105-computed-properties`



`v-if` vs. `v-show`

- There is also a `v-show` directive in Vue
  - `v-show` – or `:hidden` - the element is rendered in the DOM, but hidden afterwards
  - `v-if` – the element is not rendered at all-in the DOM
- Directive `v-show` does NOT have a `v-else`
- Documentation:

"Generally speaking, v-if has higher toggle costs while v-show has higher initial render costs.

So prefer v-show if you need to toggle something very often, and prefer v-if if the condition is unlikely to change at runtime."

https://vuejs.org/guide/essentials/conditional.html#conditional-rendering

Binding to images

We need a bit of WebPack magic here…

Peter Kassenaar
INCLUSIEF GRATIS WEBVERSIE VAN HET BOEK
Vue.js
Web Development Library
VANDUUREN MEDIA

P. 111 e.v.

Dynamically binding to images

- Vue can not simply interpolate the name of an external resource and re-use it for binding
  - For instance, the `src` attribute of an image.
- So this is invalid:

```
<li class="list-group-item">
    <img :src="./assets/{{selectedCountry.img}}">
</li>
```

Invalid!

Compiler will throw an error

```
[plugin:vite:vue] Error parsing JavaScript expression: Unexpected token (1:1)

C:/Users/Gebruiker/Desktop/vue-project/src/components/VacationPicker.vue:26:22
24 |  <!--        Binding an image-->
25 |          <li class="list-group-item">
26 |            <img :src="../assets/{{selectedCountry.img}}" alt="">
   |                        ^
27 |          </li>
28 |          <li class="list-group-item" v-if="isExpensive">

    at createCompilerError (C:\Users\Gebruiker\Desktop\vue-project\node_modules\@vue\compiler-core\di
    at emitError (C:\Users\Gebruiker\Desktop\vue-project\node_modules\@vue\compiler-core\dist\compile
```

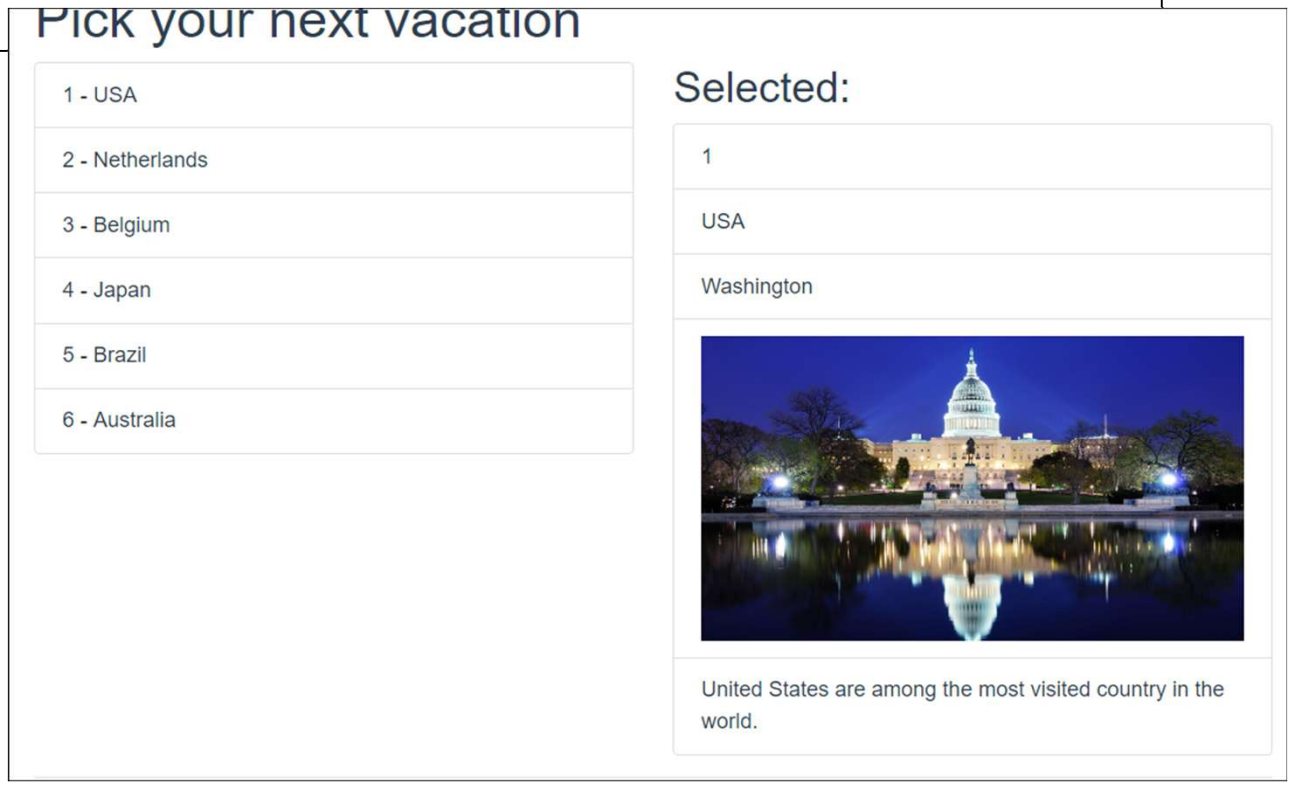
Vite composition of strings

- Because Vite builds JavaScript strings of everything, it needs to be able to determine the location of the requested file
- Create a computed property that returns a string with the correct location:

```
<li class="list-group-item">
    <img :src="imgUrl"
         class="img-fluid" >
</li>
```

```
// A computed property that returns the URL to the image for currentCountry.
const imgUrl = computed(() =>{
  return new URL(`/src/assets/countries/${currentCountry.value.img}`,
                 import.meta.url).href;
})
```

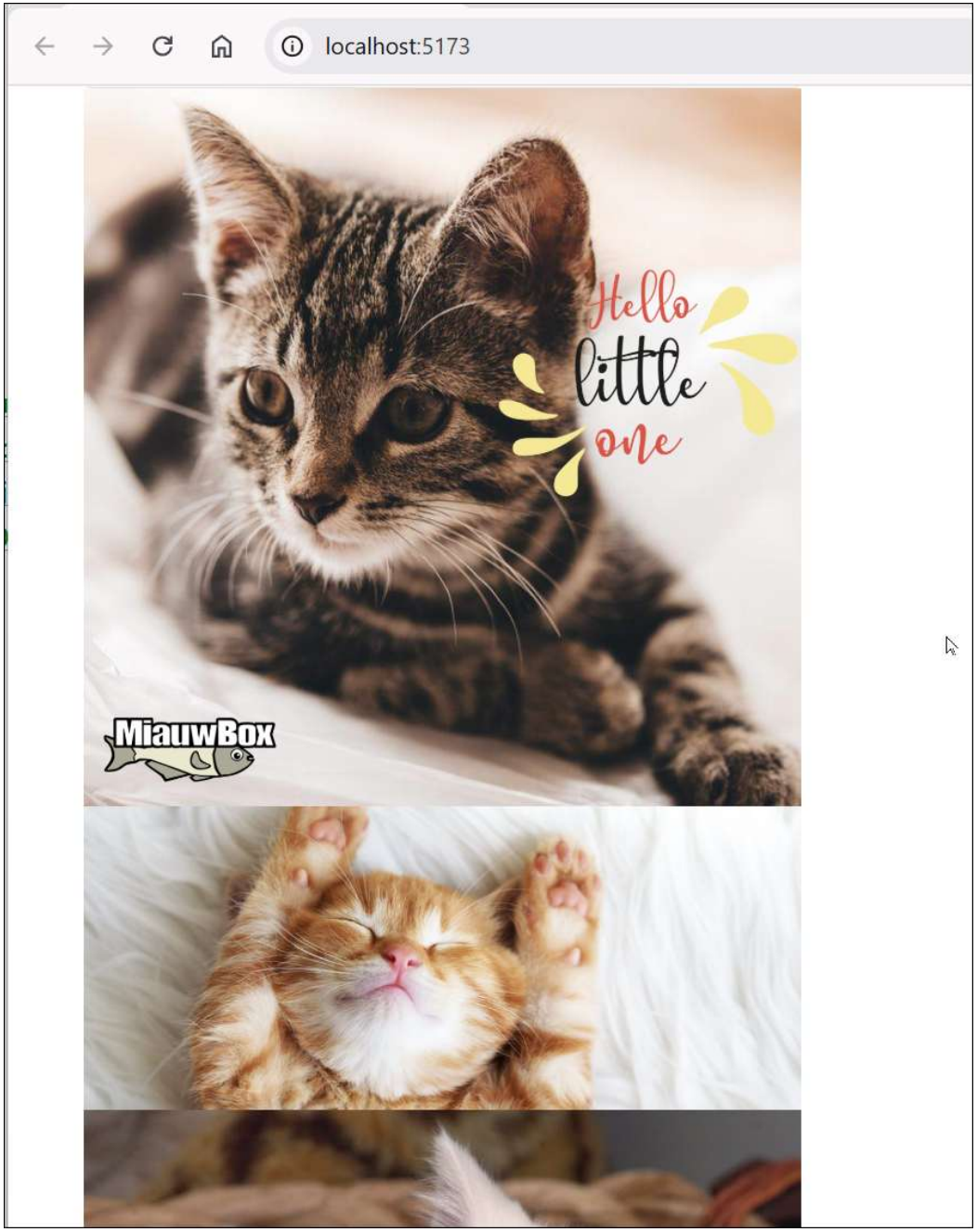
(Yes, this looks a bit overcomplicated and should have been simpler, IMO)



Binding to absolute paths / images

- Just use an array holding the absolute paths and loop over the array.
  - This is very simple

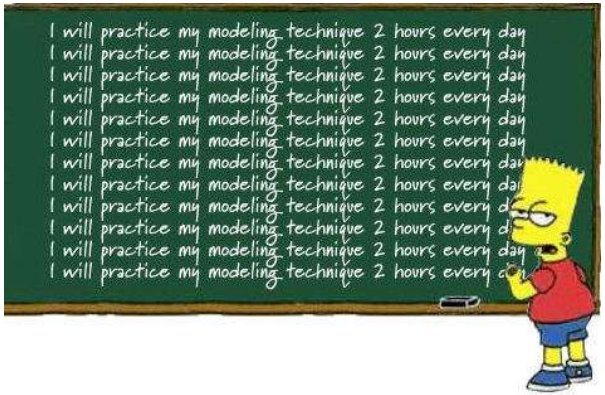
```
<template>
  <div>
    <img v-for="(kitten, index) in kittens"
         :key="index"
         class="img-fluid" :src="kitten" alt="Kitten!">
  </div>
</template>

<script setup>
const kittens = [
  "https://www.miauwbox.nl/.../Miauwbox-website-kittenbox.jpg",
  "https://www.beeztees.nl/.../shutterstock_235089946-1000x423.jpg",
  "https://www.licg.nl/.../kittenchecklist740x433.jpg",
  "https://www.operationkindness.org/.../nursery-operation-kindness.jpg"
]
</script>
```


localhost:5173
Hello
little
one
MiauwBox

Workshop

1. Static / Absolute:
   - Create an array of images in your app, – has to be a complete URL!
   - Render them in a `v-for` loop. This can be done at any time.
   - See for instance https://www.youtube.com/watch?v=B8rVlxQm8Cs.
2. Dynamic / Relative
   - Add/use an image as property of an object (like with the countries)
   - Render them conditionally/dynamically in the UI, like in the previous slides
   - Use the Vite `composition` function.

- Example: `../106-image-binding`

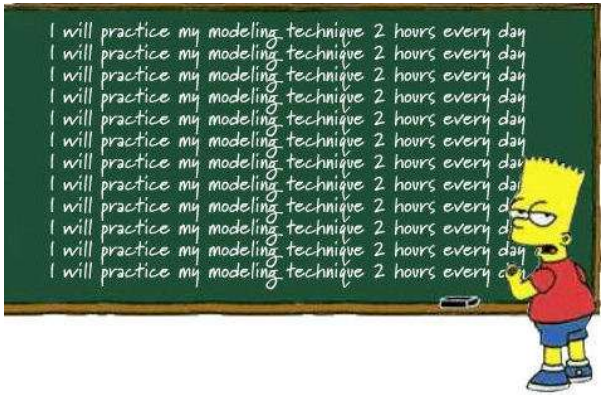


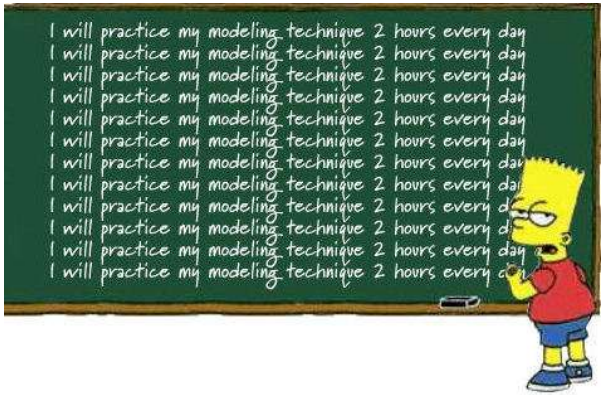
Checkpoint

- You can import and use third party libraries
- You know the most important data binding concepts of Vue
- `{{ .. }}` for simple data binding expressions
- `v-bind:` or simply `:` for attribute binding
- `v-on:` or simply `@` for event binding
- `v-if` for conditional rendering
- Computed properties that update on a triggered change
- Some WebPack stuff to render images in the UI

Final Workshop - 1

- Create the app shown in the beginning of the slides
  - One country is visible at a time
  - Its name and capital are shown
  - There are two buttons to cycle through the countries
  - User can hide/show details with a button
  - There are badges if visiting the country is Expensive, or On Sale
- Example: `../110-data-binding-wrap-up`
  - But first try it yourself



Final Workshop – 2

- Create the same app as before, but in TypeScript
  - Start from scratch, choose `TypeScript` as an option
  - Create your components as TypeScript classes
  - Create properties, methods, computed properties (getters) and load the data.
- No Example available